Reviewer Pack — Minimal Rank of Tropicalized Noncrossing Partitions vs BP_Re…

Entry 1e87f5d63dae · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 09:40:13 UTC

Minimal Rank of Tropicalized Noncrossing Partitions vs BP_ReadTwice Circuit Size

Entry ID: `1e87f5d63dae` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 09:40:13 UTC

1. Conjecture

Field A (mathematical branch): Combinatorial Enumeration (Noncrossing Partitions) **Field B** (complexity object): Complexity Theory: BP_ReadTwice Circuit Complexity

Statement:

> ['For a given boolean function f, the minimal rank of its tropicalized noncrossing partition polynomial is Θ(log n), where n is the number of variables in f.', 'If there exists a boolean function f such that the minimal rank of its tropicalized noncrossing partition polynomial is o(log n), then for all instances with this property, the BP_readtwice circuit size of f is at least 2^n.']

Rationale (proposer's reasoning):

> ['Noncrossing partitions are a classical combinatorial object that have been used in the study of enumeration and representation theory. Tropicalizing noncrossing partitions allows us to explore their relationship with tropical algebra and its connection to complexity theory.', 'Tropicalization provides a bridge between discrete mathematics and real analysis, which may reveal new insights into circuit complexity. If the minimal rank of the tropicalized noncrossing partition polynomial is indeed logarithmic for most boolean functions, it suggests that BP_readtwice circuits are required for efficient computation.']

Taxonomy category: `BP_READTWICE` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `290b1dc46b7f1ff6`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if, for at least 80% of randomly generated boolean functions with n variables, the minimal rank of their tropicalized noncrossing partition polynomial is less than or equal to log(n) AND the BP_readtwice circuit size is strictly greater than or equal to 2^n. The conjecture is falsified if any seed generates a function where the minimal rank is greater than log(n) OR the BP_readtwice circuit size is less than 2^n.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - tropicalization AND noncrossing partitions AND rank 0(log n) AND BP_ReadTwice complexity - noncrossing partition polynomial AND minimal rank AND tropicalized AND BP_readtwice circuit size - BP_ReadTwice circuit complexity threshold AND log n AND noncrossing partitions tropicalization

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def tropicalized_noncrossing_partition_polynomial(f):
        n = len(f)
        if n == 1:
            return f[0]
        else:
            left = tropicalized_noncrossing_partition_polynomial(f[:n//2])
            right = tropicalized_noncrossing_partition_polynomial(f[n//2:])
            return max(left, right) + min(left, right)

    def bp_readtwice_circuit_size(f):
        n = len(f)
        if n == 1:
            return 1
        else:
            left = bp_readtwice_circuit_size(f[:n//2])
            right = bp_readtwice_circuit_size(f[n//2:])
            return max(left, right) + 1

    n_values = [5, 10, 15, 20, 30, 40]
    results = []
```

```
    for n in n_values:
        f = generate_boolean_function(n)
        rank = tropicalized_noncrossing_partition_polynomial(f)
        circuit_size = bp_readtwice_circuit_size(f)

        if rank > math.log(n):
            return {
                "metric_name": "minimal_rank",
                "metric_value": rank,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"n={n}, rank={rank} > log({n})"
            }

        if circuit_size < 2**n:
            return {
                "metric_name": "bp_readtwice_circuit_size",
                "metric_value": circuit_size,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"n={n}, circuit_size={circuit_size} < 2^{n}"
            }

        results.append({
            "n": n,
            "rank": rank,
            "circuit_size": circuit_size
        })

    return {
        "metric_name": "minimal_rank",
        "metric_value": sum(result["rank"] for result in results) / len(results),
        "instances_tested": len(results),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(seed) for seed in sys.argv[1:]] or [2**i - 1 for i in range(5, 35)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    total_metric_value = sum(result["metric_value"] for result in results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results)} std=0.0 support_fraction=
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={total_metric_value/len(results)} std=0.0 support_fraction=
    else:
        for result in results:
```

```
            if not result["conjecture_holds"]:
                counterexample = result
                break

        print(f"RESULT: FALSIFIED counterexample=\"n={counterexample['n']}, rank={counterexample['
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
name': 'minimal_rank', 'metric_value': 18, 'instances_tested': 1, 'conjecture_holds': False, 'counte
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 17, 'instances_tested': 1, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 14, 'instances_tested': 1, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 15, 'instances_tested': 1, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 19, 'instances_tested': 1, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 11, 'instances_tested': 1, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 16, 'instances_tested': 1, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 21, 'instances_tested': 1, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 17, 'instances_tested': 1, 'conjecture_holds'
TRIAL: {'metric_name': 'minimal_rank', 'metric_value': 15, 'instances_tested': 1, 'conjecture_holds'

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_af5998f1.py", line 104, in <module>
    print(f"RESULT: FALSIFIED counterexample=\"n={counterexample['n']}, rank={counterexample['rank']
                                                 ~~~~~~~~~~~~~~~^^^^^
KeyError: 'n'
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Safety rail: critic_challenge_falsified | original: The conjecture has been falsified by a counterexample where for n=5, the minimal rank of the tropicalized noncrossing partition polynomial is greater | next: Investigate further to find more counterexamples or refine the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11693
2	preregistration	ollama_remote	glm4:latest	0	0	6111
3	novelty	ollama_remote	glm4:latest	0	0	4831

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
4	novelty	ollama_remote	glm4:latest	0	0	5810
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12042
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7338
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7607
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9712
9	judge	ollama_remote	glm4:latest	0	0	22610

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 87754 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/1e87f5d63dae.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/1e87f5d63dae.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/1e87f5d63dae.tar.gz` (if generated)